

Branch & Bound

Algorithm Design → Search

Discrete Maths for Engineers

Leandro Rodrigues

March 27, 2026

Contents

1	Problem Model & Search Space	4
1.1	Roadmap	4
1.2	Why start with the model?	4
1.3	Candidate solutions	4
1.4	The search space	4
1.5	Feasibility	5
1.6	Objective function	5
1.7	Maximization and minimization	5
1.8	Why this matters	5
1.9	Example problems	6
1.10	Next: Part 2 — The Core Idea (Branch + Bound + Pruning)	6
2	The Core Idea (Branch + Bound + Pruning)	7
2.1	Roadmap	7
2.2	What is “branch”?	7
2.3	What is a bound?	7
2.4	Upper bounds and lower bounds	8
2.5	The current best solution	8
2.6	What is pruning?	8
2.7	The core decision logic	8
2.8	Why pruning does not break correctness	9
2.9	Branch & Bound vs. Backtracking	9
2.10	Next: Part 3 — Generic Branch & Bound Algorithm	9
3	Generic Branch & Bound Algorithm	10
3.1	Roadmap	10
3.2	High-Level Structure	10
3.3	Generic Pseudocode	10
3.4	Node Selection Strategy	10
3.5	The Node Set (Frontier)	11
3.6	Updating the Incumbent	11
3.7	Where Pruning Happens	11
3.8	Key Properties	11
3.9	Next: Part 4 — Bounding Functions	11

4	Bounding Functions	12
4.1	Roadmap	12
4.2	What is a bounding function?	12
4.3	Safe bounds	12
4.4	Tight vs. loose bounds	12
4.5	Bound quality vs. computation cost	13
4.6	Bounding in maximization and minimization	13
4.7	Where bounds come from	13
4.8	Why bounding functions matter so much	13
4.9	Summary	14
5	Node Selection Strategies	15
5.1	Roadmap	15
5.2	Why node selection matters	15
5.3	The frontier of live nodes	15
5.4	Depth-first search (DFS)	15
5.5	Breadth-first search (BFS)	16
5.6	Best-first search	16
5.7	Comparing the main strategies	16
5.8	Interaction with the incumbent	16
5.9	Summary	16
5.10	Next: Part 6 — Classic Problems	17
6	Classic Problems	18
6.1	Roadmap	18
6.2	0/1 Knapsack	18
6.3	Traveling Salesman Problem (TSP)	18
6.4	Assignment Problems	19
6.5	Scheduling Problems	19
6.6	Integer Programming	19
6.7	The common pattern across problems	19
6.8	Why these are called “classic” problems	19
6.9	Summary	20
6.10	Next: Part 7 — Complexity	20
7	Complexity	21
7.1	Roadmap	21
7.2	Worst-case complexity	21
7.3	Why it is still useful	21
7.4	The effect of pruning	21
7.5	Space complexity	22
7.6	Cost of computing bounds	22
7.7	Instance-dependent behavior	22
7.8	Practical view of complexity	22
7.9	Summary	23
7.10	Next: Part 8 — Branch & Bound vs Backtracking vs Dynamic Programming	23
8	Branch & Bound vs Backtracking vs Dynamic Programming	24
8.1	Roadmap	24
8.2	Branch & Bound vs Backtracking	24
8.3	Branch & Bound vs Dynamic Programming	24
8.4	Where they overlap	24

8.5	When Backtracking is the right choice	25
8.6	When Branch & Bound is the right choice	25
8.7	When Dynamic Programming is the right choice	25
8.8	Comparative summary	25
8.9	Summary	26
8.10	Series complete	26

1 Problem Model & Search Space

1.1 Roadmap

- What kind of problems Branch & Bound solves
- How candidate solutions are modeled
- Why the search space forms a tree
- The role of feasibility and the objective function
- Why exponential growth requires intelligent pruning

Branch & Bound is often introduced informally as a method that explores possibilities and discards branches that cannot lead to the optimal solution.

That intuition is useful, but to understand the technique rigorously, we must first define the problem model and the search space it explores.

1.2 Why start with the model?

Branch & Bound is not tied to a single specific problem. It is a general strategy for solving combinatorial optimization problems.

Therefore, before discussing bounds, pruning, or implementation, we must answer:

- What is a candidate solution?
- How is a partial solution represented?
- What does it mean for a solution to be feasible?
- What exactly are we optimizing?

Main idea: Branch & Bound is a method for exploring a structured search space in optimization problems.

1.3 Candidate solutions

A candidate solution is typically constructed as a sequence of decisions:

$$x = (x_1, x_2, \dots, x_k)$$

Each variable x_i represents a choice made at step i , usually selected from a domain of possible values.

A complete assignment may represent a final solution, while a prefix $(x_1, \dots, x_i)$ represents a partial solution.

1.4 The search space

The set of all candidate solutions defines the search space.

This space is naturally modeled as a tree:

- The root is the empty solution
- Each level corresponds to one additional decision

- Each edge represents a choice
- Each node represents a partial assignment

In abstract terms, a node can be written as $(x_1, \dots, x_i)$.

1.5 Feasibility

Not every partial solution should be explored further.

A Branch & Bound problem includes constraints that determine whether a partial solution can still be extended into a valid full solution.

$$C(x_1, x_2, \dots, x_i)$$

If the current assignment already violates the problem structure, that branch can be stopped immediately.

First pruning idea: reject infeasible branches as early as possible.

1.6 Objective function

Unlike classical backtracking, in Branch & Bound we are not satisfied with just any valid solution: we want the best feasible solution according to an objective function.

Formally, we want to find a solution x^* such that:

$$x^* \in S \text{ and } f(x^*) \text{ is optimal}$$

where S is the set of feasible solutions and $f(x)$ measures the value of a solution.

1.7 Maximization and minimization

Branch & Bound applies to two major types of problems:

- Maximization: maximize profit, value, or score
- Minimization: minimize cost, distance, or time

The structure of the search remains the same. What changes is how we compare solutions and interpret bounds.

1.8 Why this matters

The search space often grows exponentially with the size of the input. A brute-force method would examine all possibilities.

$$O(2^n)$$

In real problems, this quickly becomes infeasible. That is why we need a strategy that preserves correctness while drastically reducing the number of explored branches.

Core motivation: exploring everything guarantees correctness, but is too expensive. Branch & Bound aims to keep correctness with selective exploration.

1.9 Example problems

This model appears in many classical problems:

- 0/1 Knapsack
- Traveling Salesman Problem
- Scheduling
- Assignment problems
- Integer programming

In all of them, the core idea is the same: build solutions incrementally, evaluate feasibility and quality, and avoid exploring regions that cannot contain the optimal solution.

1.10 Next: Part 2 — The Core Idea (Branch + Bound + Pruning)

Now that we have defined the structure of the problem and the search space, the next step is to understand the mechanism that decides which branches to explore and which to discard.

In Part 2, we will study how bounds are defined, how they guide pruning, and why this makes the search significantly more efficient.

2 The Core Idea (Branch + Bound + Pruning)

2.1 Roadmap

- What “branch” means in the search process
- What a bound is and why it matters
- How pruning decisions are made
- The role of the current best solution
- Why Branch & Bound remains exact despite pruning

In Part 1, we modeled the problem as a search tree of partial decisions. That gave us the structure.

Now we study the mechanism that makes Branch & Bound powerful: it does not explore the tree blindly. Instead, it expands promising regions and cuts off regions that cannot improve the current best solution.

2.2 What is “branch”?

The word branch refers to the act of splitting a problem into smaller subproblems by making one additional decision.

In tree form:

- each node is a partial solution,
- each outgoing edge is a possible next choice,
- each child node represents one refinement of the current state.

For example, in 0/1 Knapsack, branching on one item usually means:

- include the item, or
- exclude the item.

Main idea: branching generates the search tree by expanding partial decisions.

2.3 What is a bound?

A bound is an estimate of the best result that could still be obtained from a given partial solution.

Since a node is not yet a complete solution, we usually do not know its final value exactly. Instead, we compute a limit that tells us how promising that node is.

In abstract form, we may write:

$$B(x_1, \dots, x_i)$$

where B estimates the best achievable completion of the partial assignment $(x_1, \dots, x_i)$.

2.4 Upper bounds and lower bounds

The meaning of a bound depends on the type of optimization problem.

- In maximization, we often compute an upper bound on what the node could still achieve.
- In minimization, we often compute a lower bound on the best cost still possible.

The general principle is the same: the bound tells us whether the node still has any chance of beating the current best solution.

Interpretation: a bound is not the answer — it is a safe estimate used for decision making.

2.5 The current best solution

Branch & Bound keeps track of the best complete feasible solution found so far. This is often called the incumbent.

The incumbent gives us a concrete threshold for pruning:

- if a node can still beat the incumbent, explore it;
- if it cannot beat the incumbent, discard it.

In maximization problems, if the upper bound of a node is not better than the incumbent value, then that node is useless.

If $B(\text{node}) \leq \text{best}$, then the node can be pruned in maximization.

2.6 What is pruning?

Pruning means stopping the exploration of a branch before it reaches a leaf.

This happens when we can prove that continuing that branch is pointless. In Branch & Bound, this usually occurs for one of two reasons:

- the branch is infeasible, or
- the branch is feasible but cannot improve the incumbent.

The second reason is what distinguishes Branch & Bound from ordinary backtracking.

Core pruning principle: eliminate branches that are impossible or uncompetitive.

2.7 The core decision logic

At each node, the algorithm conceptually asks:

- Is this partial solution feasible?
- If completed optimally, could it beat the incumbent?

If the answer to the second question is no, the branch is cut immediately.

So the search follows this pattern:

1. Generate a partial solution

2. Check feasibility
3. Compute its bound
4. Compare with the incumbent
5. Expand or prune

2.8 Why pruning does not break correctness

Branch & Bound is still an exact algorithm. It does not rely on guessing or approximation alone.

Correctness is preserved because pruning is only performed when the bound proves that no completion of that branch can improve the current best known solution.

In other words, the algorithm never discards a branch that could contain a better solution.

Key guarantee: pruning is safe because it is justified by a valid bound.

2.9 Branch & Bound vs. Backtracking

The two techniques are related, but their goals differ.

- Backtracking mainly prunes branches that violate constraints.
- Branch & Bound also prunes branches that are valid, but not good enough to matter.

So Branch & Bound can be seen as:

backtracking + optimization-aware pruning

That is why it is especially useful when the goal is not merely to find a solution, but to find the optimal one.

2.10 Next: Part 3 — Generic Branch & Bound Algorithm

Now that we understand the core logic of branching, bounding, and pruning, the next step is to put these ideas into a general algorithmic structure.

In Part 3, we will study the generic Branch & Bound workflow: how nodes are created, evaluated, selected, and expanded during the search.

3 Generic Branch & Bound Algorithm

3.1 Roadmap

- The generic structure of the algorithm
- How nodes are stored and selected
- Where bounds are computed
- How pruning integrates into the flow
- Different search strategies (DFS, BFS, Best-first)

In Part 2, we introduced the core ideas: branching, bounding, and pruning.

Now we organize these ideas into a generic algorithm that can be applied to a wide range of optimization problems.

3.2 High-Level Structure

Branch & Bound explores a search tree while maintaining a set of nodes that are candidates for expansion.

At a high level, the algorithm repeatedly:

1. Selects a node
2. Evaluates its bound
3. Decides whether to prune or expand
4. Updates the best solution if needed

3.3 Generic Pseudocode

```
Initialize best <- -infinity (or +infinity for minimization)
Initialize node set with root

while node set is not empty:
    select a node N
    if N is a complete solution:
        update best if better
    else:
        compute bound B(N)
        if B(N) is promising:
            branch and add children to node set
        else:
            prune N
```

This structure captures the essence of Branch & Bound: systematic exploration guided by bounds.

3.4 Node Selection Strategy

The order in which nodes are explored has a major impact on performance.

- DFS (Depth-first): low memory, fast to find a solution
- BFS (Breadth-first): explores level by level

- Best-first: expands the most promising node first

Best-first search is often preferred because it quickly finds strong incumbents, improving pruning effectiveness.

3.5 The Node Set (Frontier)

The algorithm maintains a collection of nodes waiting to be explored. This is often called the frontier.

The data structure depends on the strategy:

- Stack → DFS
- Queue → BFS
- Priority Queue → Best-first

3.6 Updating the Incumbent

Whenever a complete feasible solution is found, it is compared with the current best.

If it improves the objective:

- update the incumbent
- tighten pruning conditions

This makes the algorithm progressively more selective.

3.7 Where Pruning Happens

Pruning occurs immediately after computing the bound.

If the bound cannot beat the incumbent → discard node

This avoids generating unnecessary children and reduces the search space dramatically.

3.8 Key Properties

- Systematic exploration of the search space
- Guided by bounds
- Pruning guarantees efficiency
- Still guarantees optimality

Insight: performance depends heavily on the quality of the bound and the node selection strategy.

3.9 Next: Part 4 — Bounding Functions

Now that we understand the algorithmic structure, the next critical component is how bounds are computed.

In Part 4, we will study bounding functions in detail, including how to design tight and efficient estimates.

4 Bounding Functions

4.1 Roadmap

- What a bounding function must do
- The difference between tight and loose bounds
- Upper bounds vs. lower bounds
- Trade-offs between bound quality and computation cost
- Why bounding functions determine practical performance

In Branch & Bound, the search is not guided by brute force alone. It is guided by estimates.

Those estimates come from bounding functions, which tell us how promising a partial solution is and whether it is still worth exploring.

4.2 What is a bounding function?

A bounding function maps a partial solution to a numerical estimate of its best possible completion.

$$B(\text{node})$$

The exact interpretation depends on the optimization problem:

- in maximization, it is often an upper bound
- in minimization, it is often a lower bound

In both cases, the purpose is the same: determine whether the node can still produce a better solution than the incumbent.

4.3 Safe bounds

A bounding function must be safe.

That means it must never underestimate what is possible in maximization, and never overestimate what is possible in minimization, if it is used for pruning.

Otherwise, the algorithm may incorrectly discard a branch that actually contains the optimal solution.

Key requirement: a bound must justify pruning without breaking correctness.

4.4 Tight vs. loose bounds

Not all valid bounds are equally useful.

- A tight bound stays close to the true best achievable value.
- A loose bound is valid, but too optimistic or too conservative.

Tight bounds enable more pruning because they distinguish promising nodes from weak ones more precisely.

Loose bounds preserve correctness, but often leave too many branches alive.

4.5 Bound quality vs. computation cost

A better bound is not always better in practice.

There is a trade-off:

- a cheap bound may be weak but fast to compute
- a strong bound may prune more, but cost more per node

Practical Branch & Bound design often depends on balancing these two effects.

Engineering insight: the best bound is the one that reduces total search time, not necessarily the mathematically strongest one.

4.6 Bounding in maximization and minimization

The pruning rule depends on the optimization direction.

In maximization:

if $\text{upperBound}(\text{node}) \leq \text{incumbent}$, prune

In minimization:

if $\text{lowerBound}(\text{node}) \geq \text{incumbent}$, prune

These rules express the same principle: discard nodes that cannot improve the best known solution.

4.7 Where bounds come from

Bounding functions are problem-specific.

They are often derived from:

- relaxations of the original problem
- greedy approximations
- simplified subproblems
- mathematical inequalities

For example, in 0/1 Knapsack, a common upper bound comes from allowing fractional items, which makes the problem easier and gives a safe optimistic estimate.

4.8 Why bounding functions matter so much

In theory, Branch & Bound is defined by branching, bounding, and pruning. In practice, its performance is often dominated by the quality of the bound.

Two algorithms with the same branching structure can behave very differently if one uses a strong bound and the other uses a weak one.

This is why bounding functions are often the most important design decision in a Branch & Bound implementation.

4.9 Summary

- Bounds estimate the best possible completion of a partial solution
- They must be safe to preserve correctness
- Tight bounds improve pruning
- Strong bounds may cost more to compute
- Practical performance depends heavily on bound design

Main takeaway: pruning is only as good as the bounding function behind it.

5 Node Selection Strategies

5.1 Roadmap

- Why node selection matters in Branch & Bound
- The main exploration strategies
- How DFS, BFS, and Best-first differ
- The trade-offs between memory, speed, and pruning
- Why the exploration order affects practical performance

In Branch & Bound, pruning depends on bounds, but performance also depends on which node is explored next.

Even with the same branching structure and the same bounding function, different node selection strategies can lead to very different search behavior.

5.2 Why node selection matters

At any point in the algorithm, several live nodes may remain in the frontier. The algorithm must decide which one to expand next.

This decision matters because:

- it affects how quickly a good incumbent is found,
- it influences how much memory is required,
- and it changes how early pruning becomes effective.

Main idea: node selection does not change correctness, but it can drastically change efficiency.

5.3 The frontier of live nodes

The algorithm keeps a set of nodes that have been generated but not yet expanded.

These nodes are often called live nodes, and the full set is called the frontier.

A node selection strategy is essentially a rule for choosing one node from this frontier.

5.4 Depth-first search (DFS)

In a depth-first strategy, the algorithm always continues along the most recently generated branch.

This is usually implemented with a stack.

- Advantage: low memory usage
- Advantage: often finds a complete solution quickly
- Disadvantage: may spend a long time in an unpromising region

DFS is especially attractive when memory is limited or when obtaining an initial incumbent early is useful.

5.5 Breadth-first search (BFS)

In a breadth-first strategy, the algorithm explores nodes level by level.

This is usually implemented with a queue.

- Advantage: systematic level-by-level exploration
- Advantage: useful for certain structural analyses
- Disadvantage: can require a large amount of memory

In large combinatorial problems, BFS is often expensive in practice because the frontier may grow very quickly.

5.6 Best-first search

In a best-first strategy, the algorithm chooses the live node with the most promising bound.

This is usually implemented with a priority queue.

- Advantage: often finds strong incumbents early
- Advantage: can improve pruning significantly
- Disadvantage: requires maintaining ordered priorities

Best-first is one of the most common strategies in practical Branch & Bound implementations.

5.7 Comparing the main strategies

- DFS: low memory, deep exploration
- BFS: wide exploration, high memory
- Best-first: exploration guided by bound quality

The best strategy depends on the problem, the quality of the bound, and the computational resources available.

Observation: a strong bound is most effective when the node selection strategy uses that information well.

5.8 Interaction with the incumbent

Node selection affects how quickly the incumbent improves.

If a strategy tends to find good complete solutions early, then the incumbent becomes strong sooner, which makes pruning more aggressive later.

This is one reason why best-first search often performs well in practice: it tends to prioritize nodes that are likely to yield strong completions.

5.9 Summary

- Node selection determines the exploration order
- DFS uses a stack
- BFS uses a queue
- Best-first uses a priority queue
- The strategy affects memory, incumbent quality, and pruning efficiency

Main takeaway: in Branch & Bound, finding the right node next is often as important as computing the right bound.

5.10 Next: Part 6 — Classic Problems

Now that we have seen how node selection affects the search process, the next step is to study the kinds of problems where Branch & Bound is classically applied.

In Part 6, we will examine standard examples such as Knapsack, Traveling Salesman, and other optimization problems where branching, bounding, and node selection come together.

6 Classic Problems

6.1 Roadmap

- Which problems are classically solved with Branch & Bound
- Why these problems fit the Branch & Bound model
- How branching, bounding, and pruning appear in each case
- What makes some problems harder than others
- How the framework adapts across different optimization settings

Branch & Bound is not tied to a single problem. It is a general framework that applies whenever we have:

- a combinatorial search space,
- an objective function,
- and useful bounds for pruning.

In this part, we examine the classical problems where Branch & Bound is most commonly studied and applied.

6.2 0/1 Knapsack

The 0/1 Knapsack problem is one of the most standard examples of Branch & Bound.

Each item may be either:

- included, or
- excluded.

This naturally creates a binary search tree.

A common bound is obtained from the fractional knapsack relaxation, where items are allowed to be taken fractionally.

Why it fits well: branching is simple, and the bound is strong and easy to compute.

6.3 Traveling Salesman Problem (TSP)

In the Traveling Salesman Problem, the goal is to find the shortest Hamiltonian cycle visiting each city exactly once.

Branch & Bound is often used by exploring partial tours and pruning those that cannot lead to an optimal complete tour.

Typical bounds may come from:

- minimum edge estimates,
- reduced cost matrices,
- or relaxations such as assignment-based bounds.

TSP is a classic example of how powerful bounds can dramatically reduce the search space.

6.4 Assignment Problems

Assignment problems ask how to match agents to tasks while minimizing cost or maximizing value.

A Branch & Bound solution typically branches on one assignment decision at a time, while the bound comes from a relaxed version of the remaining subproblem.

These problems are especially useful for understanding how lower bounds can guide minimization.

6.5 Scheduling Problems

Many scheduling problems can also be attacked with Branch & Bound.

Here, nodes usually represent partial schedules, and the bound estimates the best completion time or minimum remaining cost.

Scheduling is a good example of how the same framework adapts to domain-specific constraints and objectives.

6.6 Integer Programming

Branch & Bound is one of the foundational techniques behind modern integer programming solvers.

In this setting, the algorithm branches on integer variables and uses relaxations of the linear program to compute bounds.

This shows Branch & Bound at a more advanced level: it becomes a general engine for exact optimization.

6.7 The common pattern across problems

Although these problems look different on the surface, they share the same structural pattern:

1. Represent partial solutions as nodes
2. Branch by refining one decision
3. Compute a bound on the best completion
4. Prune nodes that cannot improve the incumbent

Core idea: Branch & Bound is problem-agnostic at the framework level, but problem-specific in how bounds are designed.

6.8 Why these are called “classic” problems

These examples are considered classic because they clearly illustrate:

- combinatorial explosion,
- the need for exact optimization,
- and the value of intelligent pruning.

They also appear repeatedly in textbooks, optimization research, and real engineering systems.

6.9 Summary

- Knapsack is the standard introductory example
- TSP shows the importance of strong bounds
- Assignment and scheduling problems show minimization use cases
- Integer programming reveals the general power of the framework
- The same three ideas always appear: branch, bound, and prune

Main takeaway: Branch & Bound is best understood not as one algorithm for one problem, but as a reusable framework across many optimization problems.

6.10 Next: Part 7 — Complexity

Now that we have seen where Branch & Bound is classically used, the next question is how expensive it is in theory and in practice.

In Part 7, we will study the complexity of the method, including worst-case behavior and the practical role of pruning.

7 Complexity

7.1 Roadmap

- The worst-case complexity of Branch & Bound
- Why pruning can help dramatically in practice
- The difference between theoretical and practical complexity
- The role of bounds and node selection in performance
- Why complexity analysis is subtle for this method

Branch & Bound is often much faster than brute force in practice, but that does not mean its worst-case complexity disappears.

To understand the method properly, we must distinguish between worst-case theoretical complexity and practical search reduction through pruning.

7.2 Worst-case complexity

In the worst case, Branch & Bound may still explore the entire search tree.

That means its time complexity can remain exponential:

$$O(b^d)$$

where:

- b is the branching factor
- d is the depth of the search tree

In binary decision problems, this is often written as:

$$O(2^n)$$

7.3 Why it is still useful

If the worst case is still exponential, why use Branch & Bound at all?

Because in many practical instances, the algorithm avoids exploring most of the tree.

Strong bounds and good incumbents can cut away huge portions of the search space before they are ever expanded.

Main point: Branch & Bound does not improve the worst case, but it often improves the average practical case dramatically.

7.4 The effect of pruning

The more nodes are pruned, the smaller the effective search tree becomes.

This means practical running time depends on:

- the quality of the bounding function,
- the speed of finding strong incumbents,
- and the node selection strategy.

So two Branch & Bound implementations for the same problem can have the same worst-case complexity but very different practical behavior.

7.5 Space complexity

Space complexity depends strongly on the node selection strategy.

- DFS: usually low memory, because only a path and a small frontier are kept
- BFS: can require very large memory, because many nodes at the same level remain stored
- Best-first: often requires substantial memory, since many live nodes may stay in the priority queue

In practice, memory can be as important as time when evaluating a Branch & Bound method.

7.6 Cost of computing bounds

Complexity is not determined only by how many nodes are visited.

Each node may also require nontrivial work to compute its bound.

So total running time depends on both:

1. the number of visited nodes,
2. and the cost per node.

This is why a stronger bound is not automatically better: it may reduce the tree, but increase the work done at each node.

7.7 Instance-dependent behavior

Branch & Bound is highly instance-dependent.

On some inputs, pruning may be extremely effective. On others, almost no pruning may happen at all.

This makes precise complexity prediction difficult.

Observation: Branch & Bound is one of those algorithms where asymptotic complexity alone tells only part of the story.

7.8 Practical view of complexity

In practice, Branch & Bound is often evaluated by:

- number of expanded nodes,
- number of pruned nodes,
- memory usage,
- and total running time on representative instances.

This practical perspective is especially important in optimization engineering and solver design.

7.9 Summary

- Worst-case time remains exponential
- Pruning can drastically reduce search in practice
- Space depends strongly on the exploration strategy
- Total cost includes both visited nodes and bound computation
- Performance is highly instance-dependent

Main takeaway: Branch & Bound is not powerful because it changes exponential worst-case complexity, but because it often makes exact search practically feasible.

7.10 Next: Part 8 — Branch & Bound vs Backtracking vs Dynamic Programming

Now that we understand the complexity of Branch & Bound, the final step is to position it relative to other major techniques.

In Part 8, we will compare Branch & Bound with Backtracking and Dynamic Programming, highlighting where each one is most appropriate.

8 Branch & Bound vs Backtracking vs Dynamic Programming

8.1 Roadmap

- How Branch & Bound differs from Backtracking
- How it differs from Dynamic Programming
- When the three methods overlap
- When each method is most appropriate
- How to choose the right tool for a problem

Branch & Bound is often introduced near Backtracking because both explore search trees.

It is also often compared with Dynamic Programming because all three methods are used to solve difficult combinatorial problems.

But their goals, assumptions, and mechanisms are different.

8.2 Branch & Bound vs Backtracking

Backtracking is mainly concerned with feasibility.

It explores a search tree and prunes branches when constraints are violated or when no valid completion is possible.

Branch & Bound goes further: it may also prune branches that are still feasible, but cannot lead to a solution better than the incumbent.

- Backtracking: prune because the branch is invalid
- Branch & Bound: prune because the branch is uncompetitive

Main distinction: Backtracking focuses on validity; Branch & Bound focuses on optimality.

8.3 Branch & Bound vs Dynamic Programming

Dynamic Programming is based on overlapping subproblems and optimal substructure.

Instead of exploring a search tree with pruning, it solves subproblems systematically and stores their results for reuse.

Branch & Bound, by contrast, explores a search tree of partial solutions and reduces work through pruning, not memoization.

- Dynamic Programming: avoid recomputation
- Branch & Bound: avoid exploring irrelevant regions

8.4 Where they overlap

These methods are not always completely separate.

For some optimization problems:

- Backtracking can be extended into Branch & Bound
- Dynamic Programming can solve the same problem more efficiently when its assumptions hold
- Branch & Bound may still be preferred if exact DP is too large or memory-intensive

So the right method depends not only on the problem, but also on the structure available to exploit.

8.5 When Backtracking is the right choice

Backtracking is a natural choice when:

- the goal is to find any valid solution,
- or enumerate all valid solutions,
- and pruning is mainly based on constraints.

Typical examples include:

- N-Queens
- Sudoku
- Permutation generation
- Constraint satisfaction problems

8.6 When Branch & Bound is the right choice

Branch & Bound is natural when:

- the problem is an optimization problem,
- the search space is combinatorial,
- and useful bounds can eliminate large regions of the tree.

It is especially valuable when exact optimality matters, but full exhaustive search is too expensive.

8.7 When Dynamic Programming is the right choice

Dynamic Programming is often the best choice when:

- the problem has overlapping subproblems,
- optimal substructure is present,
- and the state space is manageable enough to store.

In such cases, DP may outperform Branch & Bound because it transforms exponential search into structured computation over reusable states.

8.8 Comparative summary

- Backtracking: search + feasibility pruning
- Branch & Bound: search + optimality pruning
- Dynamic Programming: state reuse + recurrence structure

In short:

- Backtracking asks: can this branch still work?
- Branch & Bound asks: can this branch still win?
- Dynamic Programming asks: have we already solved this state?

8.9 Summary

- Backtracking is about validity
- Branch & Bound is about optimality
- Dynamic Programming is about reuse of overlapping work
- The three methods may solve related problems
- The right choice depends on the structure of the problem

Main takeaway: these methods are best seen not as competitors, but as different ways of exploiting problem structure.

8.10 Series complete

We have now covered the full Branch & Bound series: from problem modeling and search trees to bounding functions, node selection, complexity, and comparisons with other algorithmic paradigms.

The next natural step is to study a concrete implementation, usually starting with 0/1 Knapsack or Traveling Salesman.

Source Pages

- [Part 1 — Problem Model & Search Space](#)
- [Part 2 — The Core Idea \(Branch + Bound + Pruning\)](#)
- [Part 3 — Generic Branch & Bound Algorithm](#)
- [Part 4 — Bounding Functions](#)
- [Part 5 — Node Selection Strategies](#)
- [Part 6 — Classic Problems](#)
- [Part 7 — Complexity](#)
- [Part 8 — Branch & Bound vs Backtracking vs Dynamic Programming](#)